Лабораторная работа №4

Продвинутое использование git

Лащиков Алексей Антонович

2026-02-28

Российский университет дружбы народов, Москва, Россия

Докладчик

- Лащиков Алексей Антонович
- НКАбд-04-25
- Российский университет дружбы народов
- 1032253527@rudn.ru

Цель и задачи

Цель: Получение навыков правильной работы с репозиториями git.

Задачи:

- Выполнить работу для тестового репозитория.
- Преобразовать рабочий репозиторий в репозиторий с git-flow и conventional commits.

Gitflow: идея

Gitflow — модель ветвления для проектов, где важны релизы и версия.

- master — официальная история релизов (теги версий)
- develop — интеграционная ветка разработки
- feature/* — разработка новых функций
- release/* — подготовка релиза
- hotfix/* — срочные исправления из master

Gitflow: базовые команды

Инициализация:

```
git flow init
git branch
```

Feature:

```
git flow feature start feature_branch
git flow feature finish feature_branch
```

Release:

```
git flow release start 1.0.0
git flow release finish 1.0.0
```

Conventional Commits

Conventional Commits — соглашение о сообщениях коммитов, чтобы:

- проще понимать историю;
- автоматически строить CHANGELOG;
- увязать коммиты с SemVer.

Conventional Commits: формат

Структура:

```
<type>(<scope>): <subject>

<body>

<footer>
```

Примеры типов:

- `fix:` — коммит типа fix исправляет ошибку в вашем коде.
- `feat:` — коммит типа feat добавляет новую функцию в ваш код.
- `BREAKING CHANGE:` — коммит, который добавляет изменения, нарушающие обратную совместимость вашего API.
- также часто: `docs:`, `chore:`, `refactor:`, `test:`.

Ход работы

В работе я собрал тестовый репозиторий и перевёл его на связку gitflow и conventional commits, чтобы получить релизный процесс: отдельные ветки под разработку релизы, теги версий и автоматический CHANGELOG.

Подготовка окружения

Сначала установил нужные инструменты: git-flow для модели ветвления и Node.js + pnpm для утилит, связанных с conventional commits и журналом изменений (см. рис.1).

```
aalashkov@aalashkov:/tmp/gitflow$ sudo dnf install gitflow
Обновление и загрузка репозиториев:
 Copr repo for gitflow owned by elegos                                                    100% | 889.0   B/s |   2.4 KiB |  00m03s
Репозитории загружены.
Пакет                          Арх.    Версия                          Репозиторий                                  Размер
Установка:
 gitflow                       x86_64  1.12.3-1.fc34                   copr:copr.fedorainfracloud.org:elegos:gitflow    261.7 KiB

Сводка транзакции:
 Установка:        1 пакета

Общий размер входящих пакетов составляет 57 KiB. Необходимо загрузить 57 KiB.
После этой операции будут использоваться дополнительные 262 KiB (установка 262 KiB, удаление 0 B).
Is this ok [y/N]: y
[1/1] gitflow-0:1.12.3-1.fc34.x86_64                                                      100% |  40.6 KiB/s |  57.4 KiB |  00m01s
--------------------------------------------------------------------------------------------------------------------------------
[1/1] Всего                                                                               100% |  40.5 KiB/s |  57.4 KiB |  00m01s
[1/2] https://download.copr.fedorainfracloud.org/results/elegos/gitflow/pubkey.gpg   777% [<=>                ] |   0.0   B/s |   0.0   B |  00m00s
[1/2] https://download.copr.fedorainfracloud.org/results/elegos/gitflow/pubkey.gpg   777% [<=>                ] |   0.0   B/s |   0.0   B |  00m00s
[1/2] https://download.copr.fedorainfracloud.org/results/elegos/gitflow/pubkey.gpg   777% [ <=>               ] |   0.0   B/s |   0.0   B |  00m01s
[1/2] https://download.copr.fedorainfracloud.org/results/elegos/gitflow/pubkey.gpg   777% [ <=>               ] |   0.0   B/s |   0.0   B |  00m01s
[1/2] https://download.copr.fedorainfracloud.org/results/elegos/gitflow/pubkey.gpg                100% |   1.4 KiB/s | 998.0   B |  00m01s
--------------------------------------------------------------------------------------------------------------------------------
[2/2] Всего                                                                               100% |  40.5 KiB/s |  57.4 KiB |  00m01s
Импортирование ключа OpenPGP 0x98FC3AA3:
 User ID     : "elegos_gitflow (None) <elegos@gitflow@copr.fedorahosted.org>"
 Отпечаток:  935798B049C5C5C36CF30A22823CA8E6B8F63AA3
 Из          : https://download.copr.fedorainfracloud.org/results/elegos/gitflow/pubkey.gpg
Is this ok [y/N]: y
Ключ был успешно импортирован.
[1/3] Проверить файлы пакета                                                              100% | 250.0   B/s |   1.0   B |  00m00s
[2/3] Подготовить транзакцию                                                              100% |   4.0   B/s |   1.0   B |  00m00s
[3/3] Установка gitflow-0:1.12.3-1.fc34.x86_64                                                   100% | 746.6 KiB/s | 271.0 KiB |  00m00s
Завершено!
aalashkov@aalashkov:/tmp/gitflow$
```

Рис. 1: Установка и настройка инструментов

Настройка коммитов и журнала изменений

Дальше настроил:

- commitizen для удобного ввода сообщений коммитов по Conventional Commits,
- standard-changelog для генерации CHANGELOG.md на основе истории коммитов.

Рис. 2: Настройка conventional commits и changelog

Gitflow в репозитории

После этого инициализировал git-flow и привёл репозиторий к схеме master/develop, чтобы дальше делать релизы.

```
aalashikov@aalashikov:~/git-extended$ git flow init

Which branch should be used for bringing forth production releases?
   - master
Branch name for production releases: [master]
Branch name for "next release" development: [develop]

How to name your supporting branch prefixes?
Feature branches? [feature/]
Bugfix branches? [bugfix/]
Release branches? [release/]
Hotfix branches? [hotfix/]
Support branches? [support/]
Version tag prefix? [] v
Hooks and filters directory? [/home/aalashikov/git-extended/.git/hooks]
aalashikov@aalashikov:~/git-extended$
```

Рис. 3: Инициализация git-flow

Релизы и публикация

Затем выполнил релизный цикл:

Создание release-ветки, обновление CHANGELOG, завершение релиза, пуш веток и тегов, релиз на GitHub.

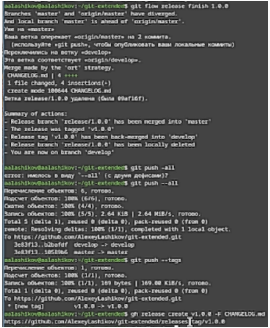

Рис. 4: Релиз и публикация

Проверка рабочего сценария

В конце прогнал типовой процесс:

Работа через feature-ветку и выпуск следующей версии.

```
aalashikov@aalashikov:~/git-extended$ git flow release start 1.2.3
Переключились на новую ветку «release/1.2.3»

Summary of actions:
- A new branch 'release/1.2.3' was created, based on 'develop'
- You are now on branch 'release/1.2.3'

Follow-up actions:
- Bump the version number now!
- Start committing last-minute fixes in preparing your release
- When done, run:

    git flow release finish '1.2.3'

aalashikov@aalashikov:~/git-extended$ micro package.json
aalashikov@aalashikov:~/git-extended$ standard-changelog
aalashikov@aalashikov:~/git-extended$ git add CHANGELOG.md
aalashikov@aalashikov:~/git-extended$ git commit -am 'chore(site): update changelog'
[release/1.2.3 79601c3] chore(site): update changelog
 2 files changed, 5 insertions(+), 1 deletion(-)
```

Рис. 5: Feature и следующий релиз

Преобразование рабочего репозитория

По заданию преобразовал основной репозиторий для загрузки лабораторных работ в репозиторий с git-flow и conventional commits.

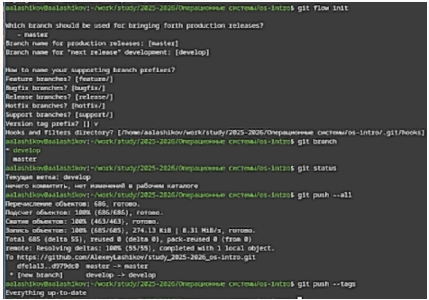

Рис. 6: Выполнение git flow init, настройка master и develop, префиксов веток и префиксв тегов v. Проверка, появления веток и отправка на GitHub

Выводы

В ходе данной лабораторной работы я:

- Разобрал модель Gitflow и назначение веток master, develop, feature, release, hotfix.
- Настроил conventional commits и генерацию CHANGELOG.
- Выполнил практический сценарий с релизами и тегами.
- Перенёс рабочий репозиторий в формат git-flow и conventional commits.